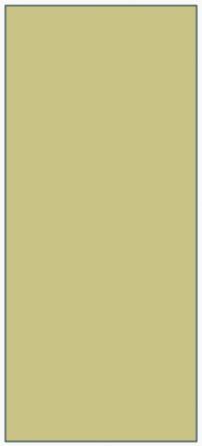
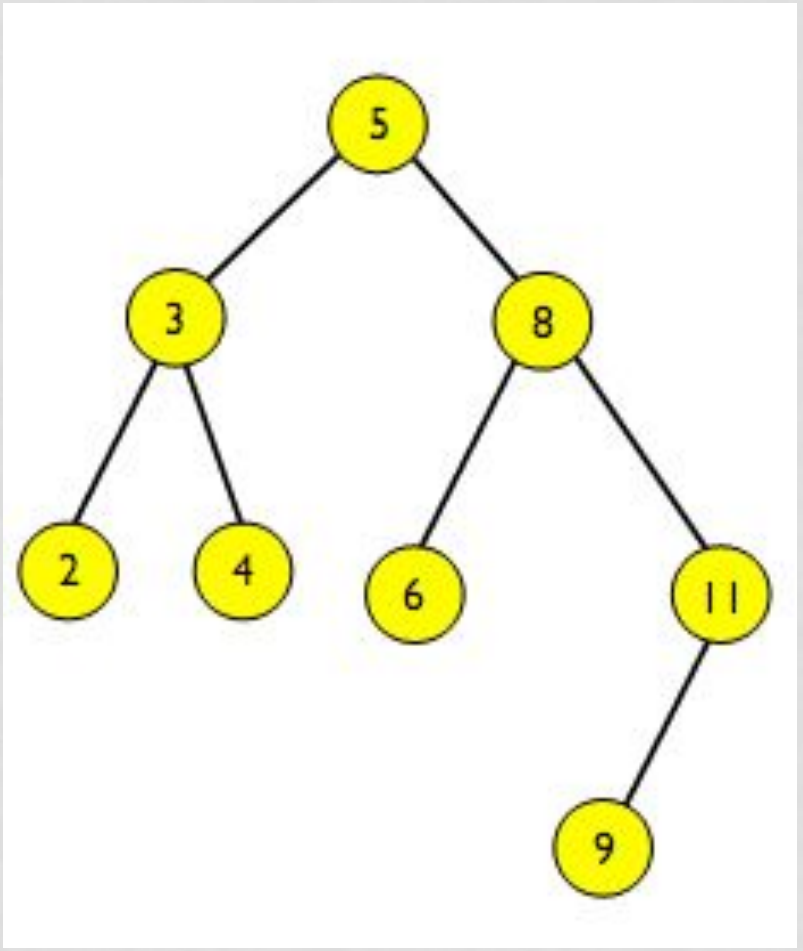
EC-200 DATA STRUCTURES

LECTURE 12

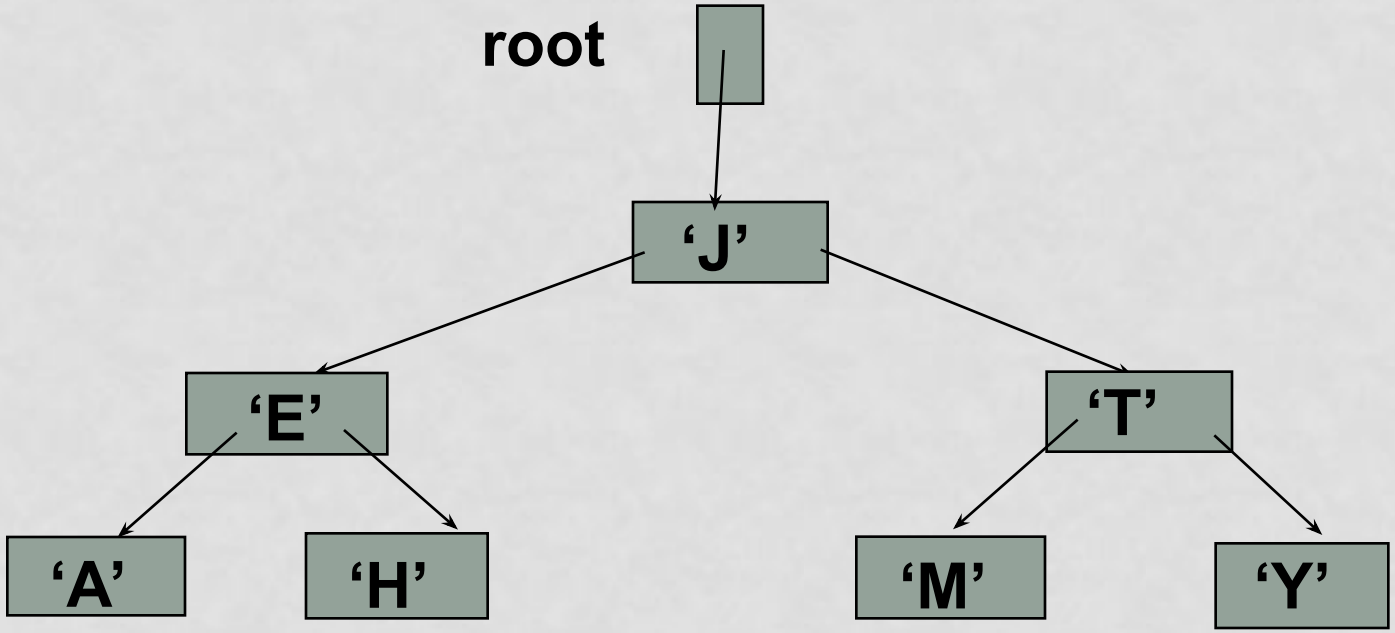


BINARY SEARCH TREE

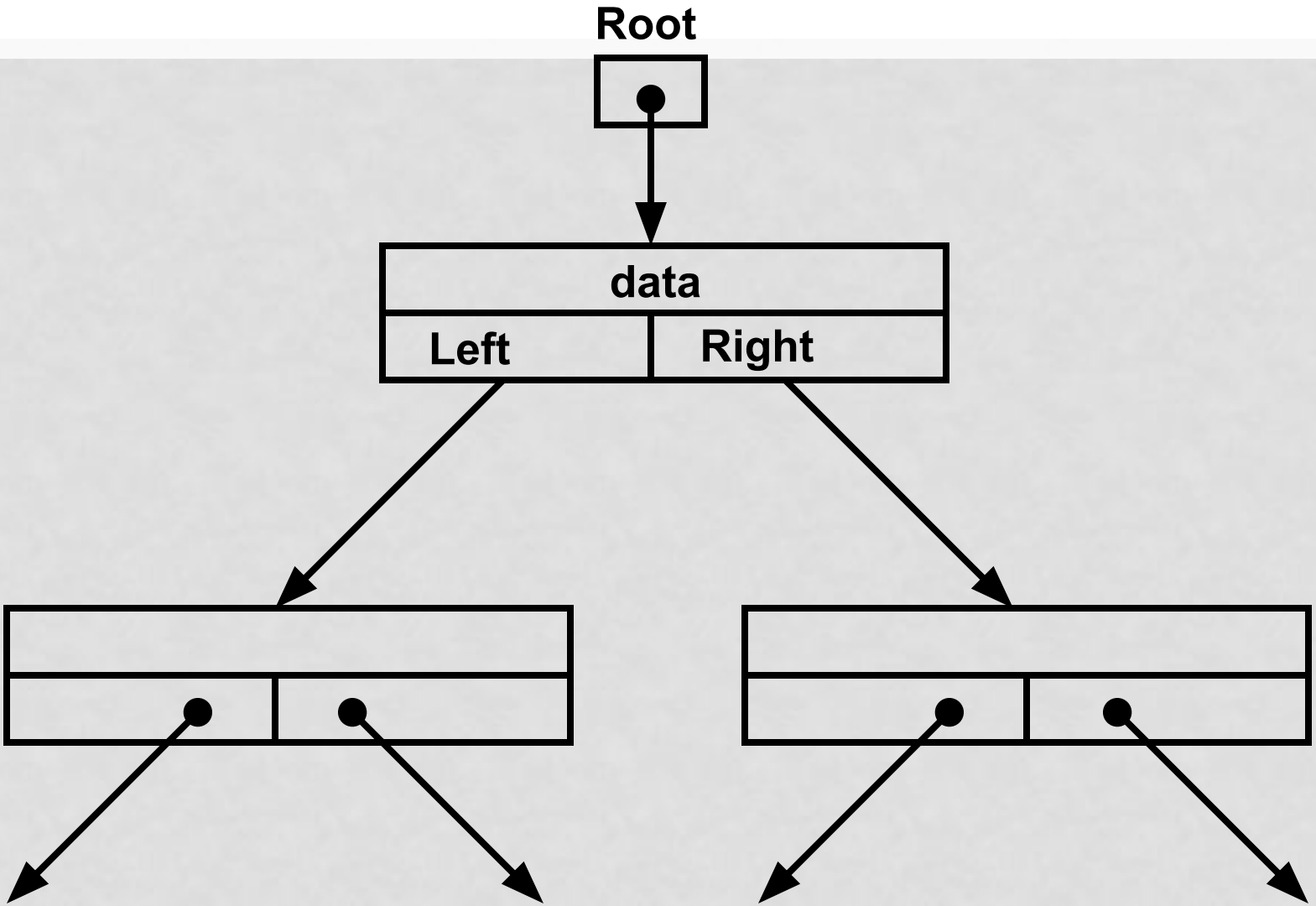
- A particular form of binary tree, such that:
- For any node in the tree containing data value x:
  - The right subtree has data values > x
  - The left subtree has data values < x



EXAMPLE BINARY SEARCH TREE



POINTER-BASED IMPLEMENTATION OF A BINARY (SEARCH) TREE

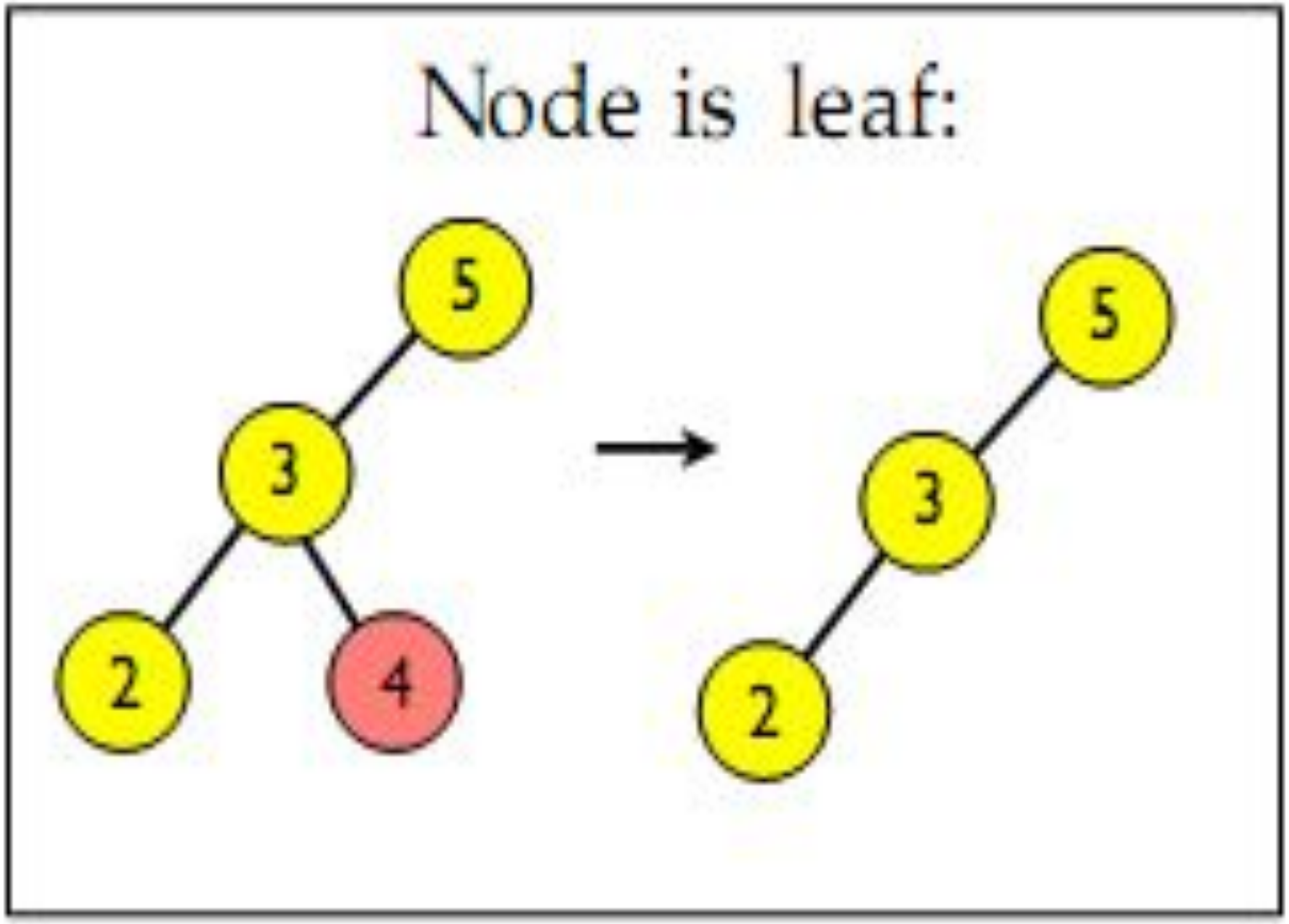


OPERATIONS IN BST

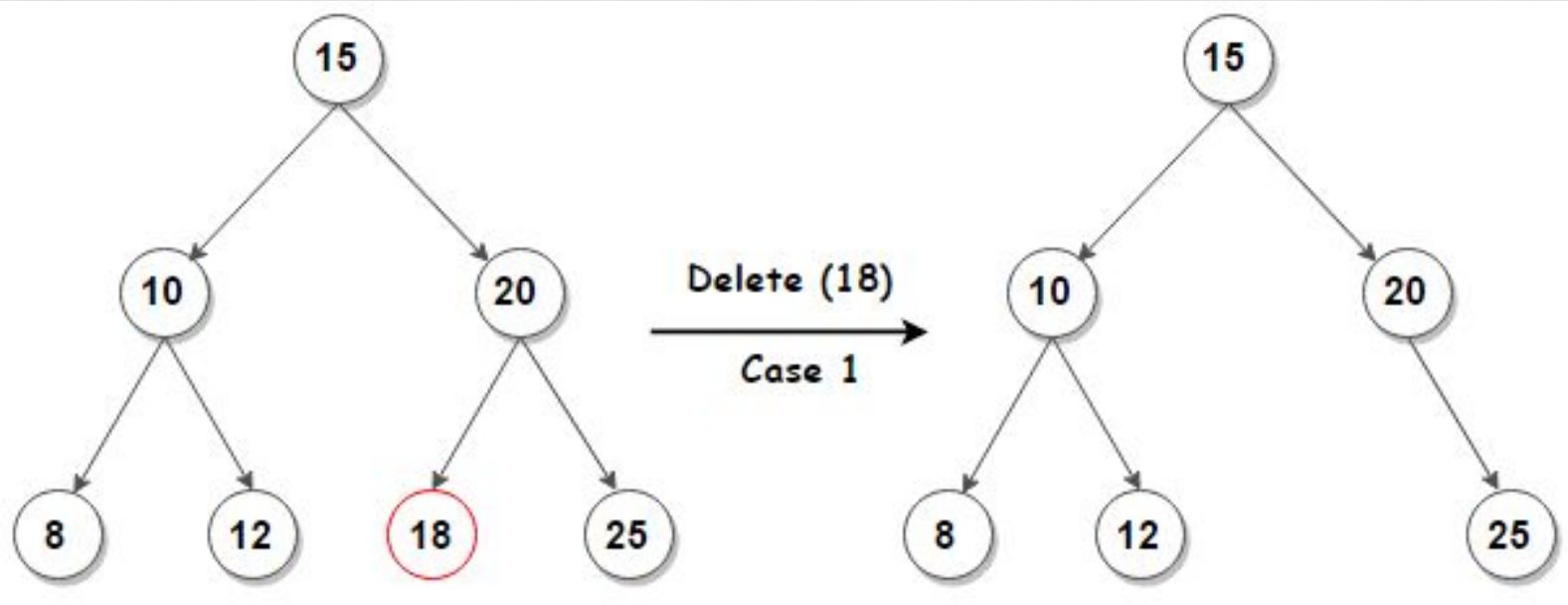
- Insert
- Search
- Traverse
- Delete

DELETION FROM A BST

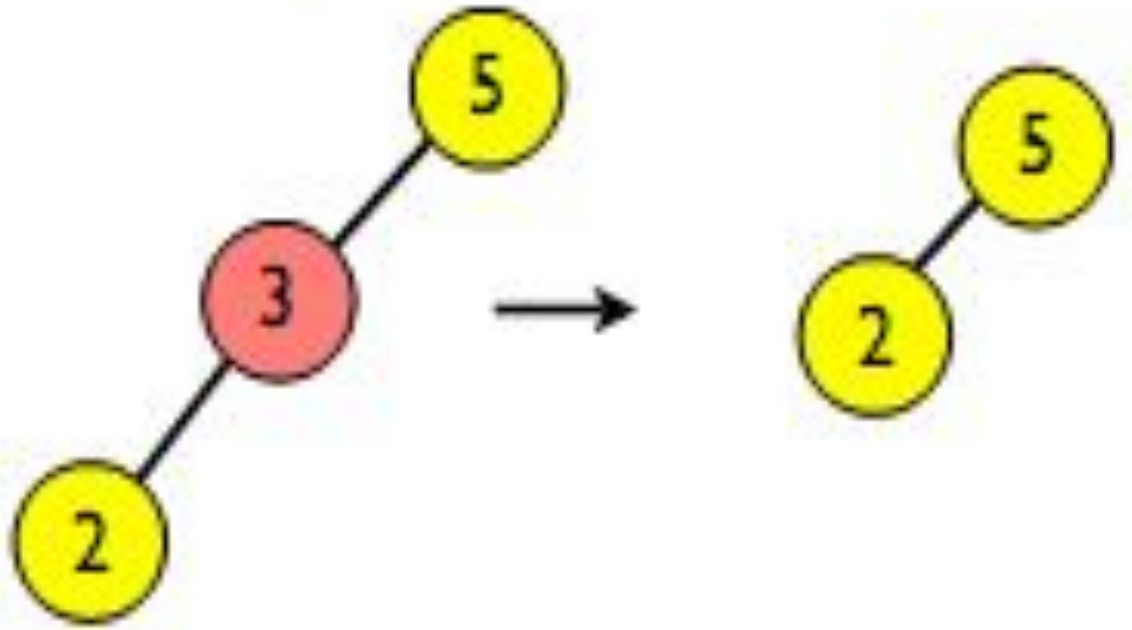
- Three cases
  - Deleting a leaf
    - No adjustment needed
  - Deleting a node with one child
    - The only child can be moved up to take the deleted node's place
  - Deleting a node with two children
    - Replace the node with its inorder successor


Node is leaf:
5
3
2
4
5
3
2

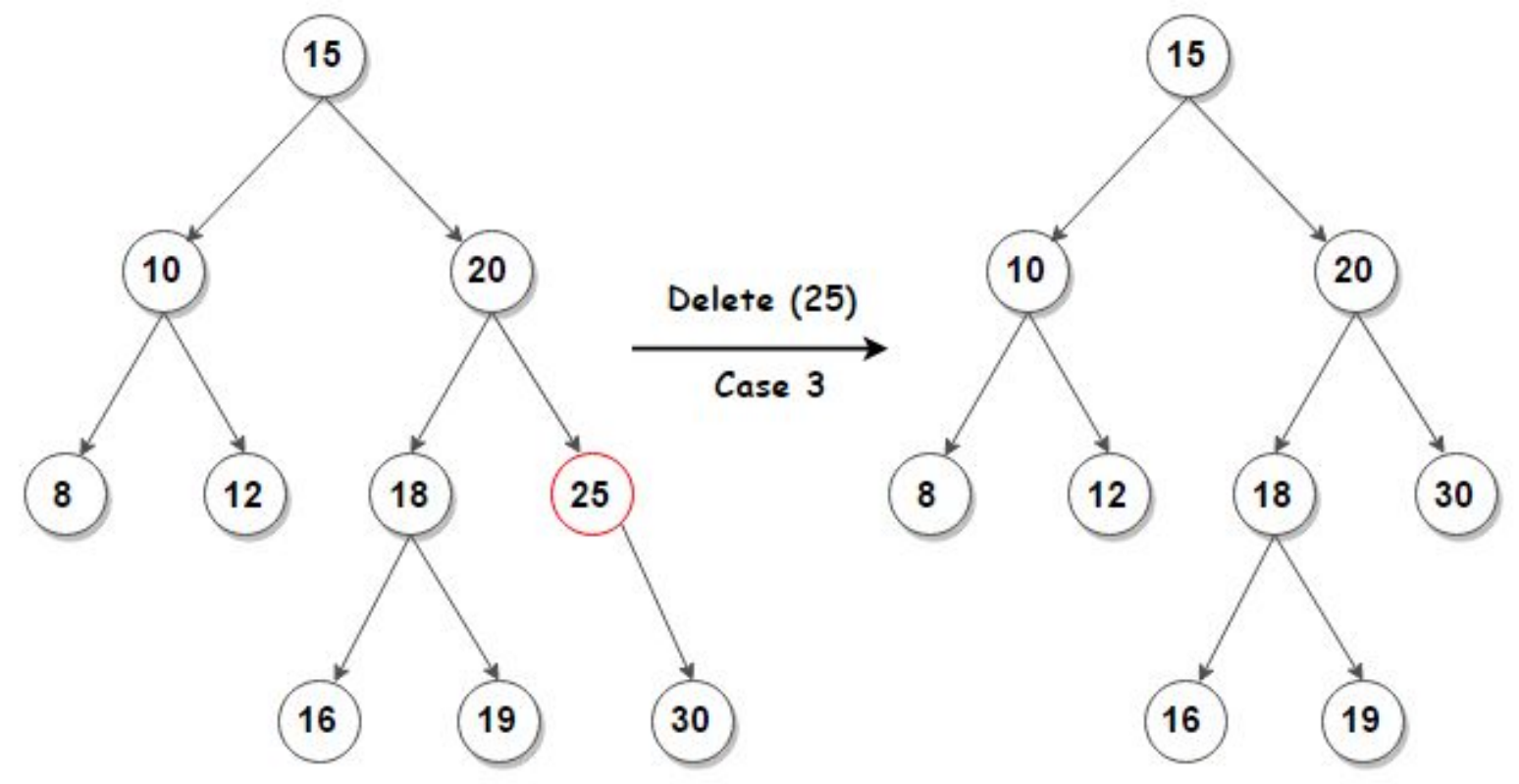
CASE 1

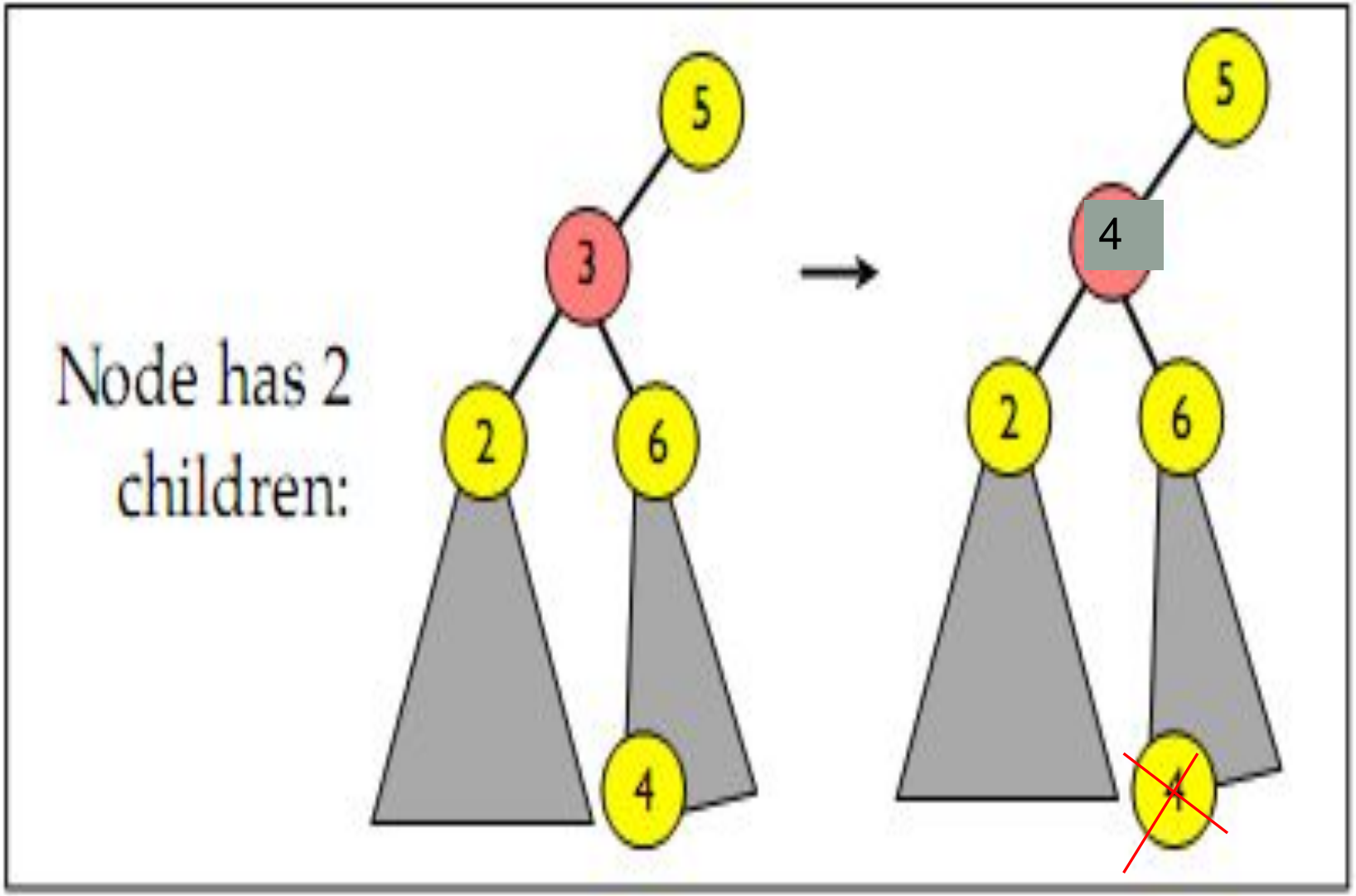


Node has 1 child:

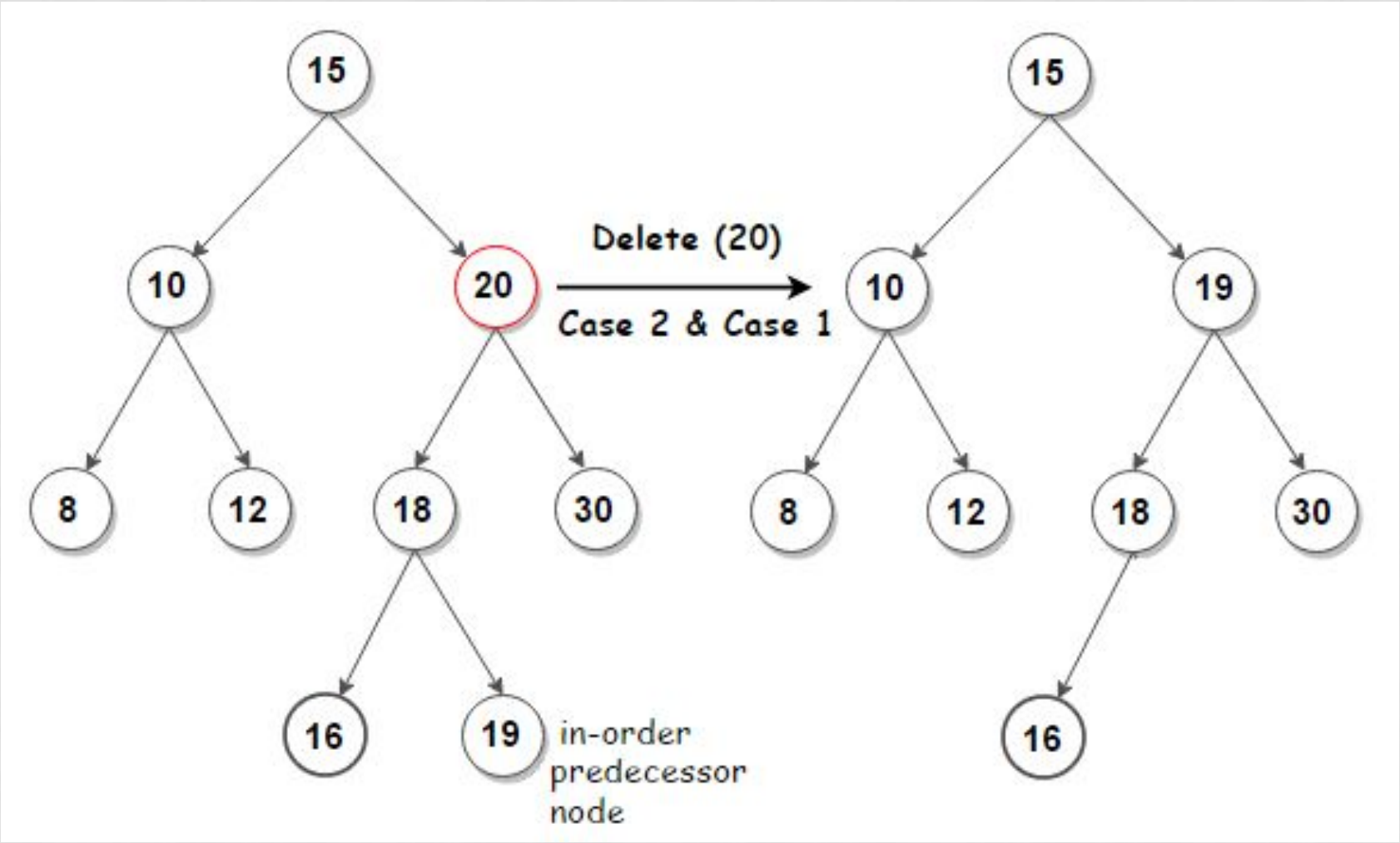


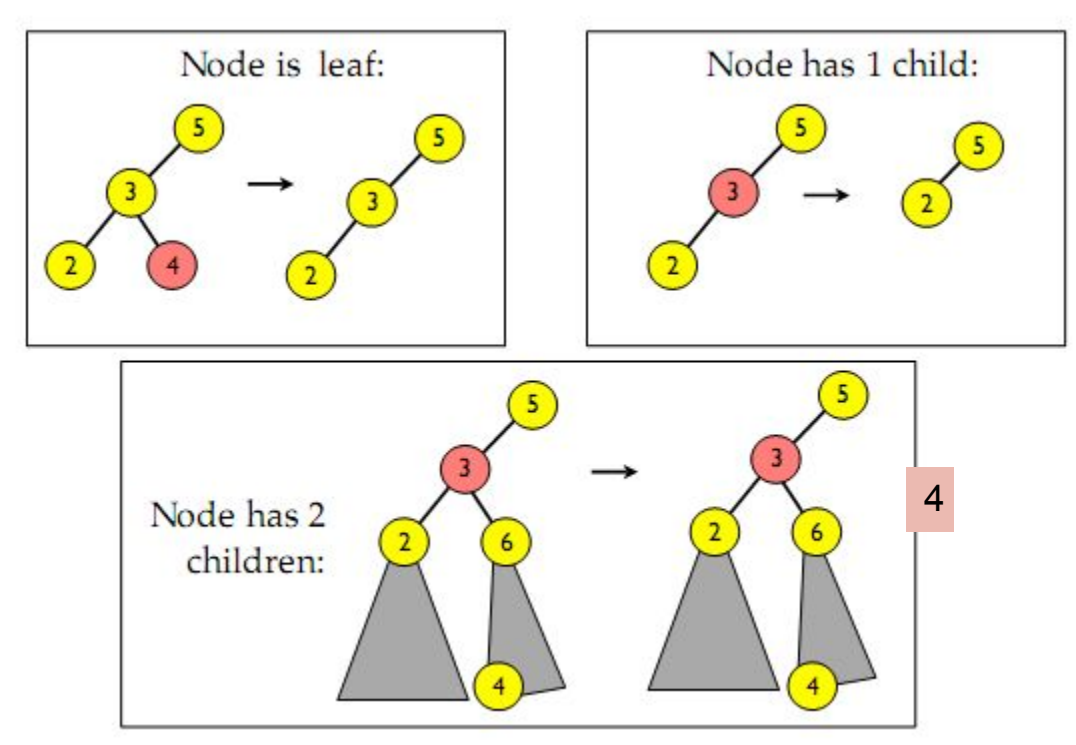
CASE 2




Node has 2 children:
5
3
2
6
4
5
4
2
6
4

CASE 3


15
10
20
8
12
18
30
16
19
in-order
predecessor
node
Delete (20)
Case 2 & Case 1
15
10
19
8
12
18
30
16


Node is leaf:
Node has 1 child:
Node has 2 children:
4

BINARY SEARCH TREE: DELETION

```
p=root; q=NULL;
//Search for the node with data x, set p to point to the node and q //to its
father, if any
while (p!=NULL && p->data!=x)
{
      q=p;
      p=(x<p->data) ? p->Left:p->Right;
}
//x does not exist in the tree, leave the tree unchanged
if (p==NULL) return;
//otherwise: set the variable rp to the node that will replace p
//first two cases: the node to be deleted has at most one child
if (p->Left==NULL) rp=p->Right;
else if (p->Right==NULL) rp=p->Left;
```

BINARY SEARCH TREE: DELETION

```
else
{
//3rd case: p has two children.
//Set rp to the inorder successor of p and f to the father of rp
f=p;
rp=p->Right;
s=rp->Left; //s is always the left child of rp
while (s!=NULL) {
      f=rp;
      rp=s;
      s=rp->Left;
      }
//at this point, rp is the inorder successor of p
```

BINARY SEARCH TREE: DELETION

```
if (f!=p)
      {
          //p is not the father of rp and rp==f->left
               f->Left=rp->Right;
          //remove node rp from its current position and replace
          //it with the right son of rp
          //rp takes the place of p
               rp->Right=p->Right;
     }
     //set the left son of rp so that rp takes the place of p
          rp->Left=p->Left;
} //end else
```

BINARY SEARCH TREE: DELETION

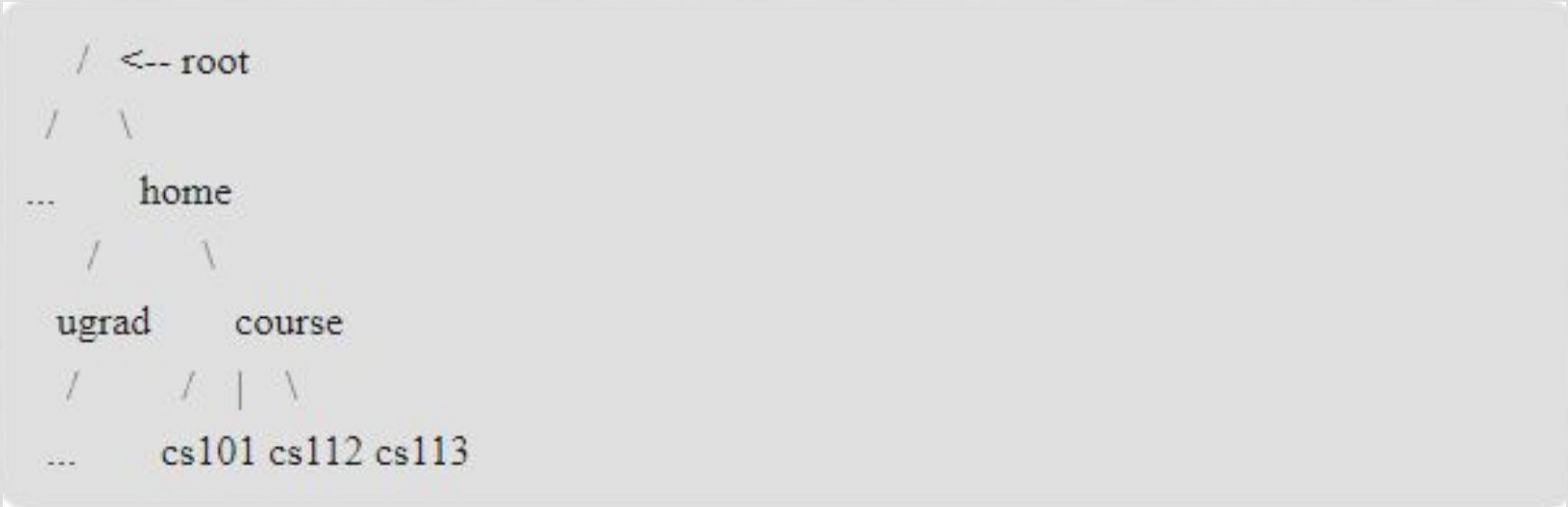
```
//insert rp into the position formerly occupied by p
if (q==NULL) //Node p was the root of the tree
    root=rp;
else
    (p==q->Left) ? q->Left=rp: q->Right=rp;
delete p;
return;
```

ADVANTAGES OF BINARY TREES

- If we organize keys in form of a tree (with some ordering e.g., BST), we can search for a given key in moderate time (quicker than Linked List and slower than arrays).
- We can insert/delete keys in moderate time (quicker than Arrays and slower than Unordered Linked Lists).
- Like Linked Lists and unlike Arrays, Pointer implementation of trees don't have an upper limit on number of nodes as nodes are linked using pointers.

APPLICATIONS OF TREES

- One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer:



READING

- Chapter 10, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.